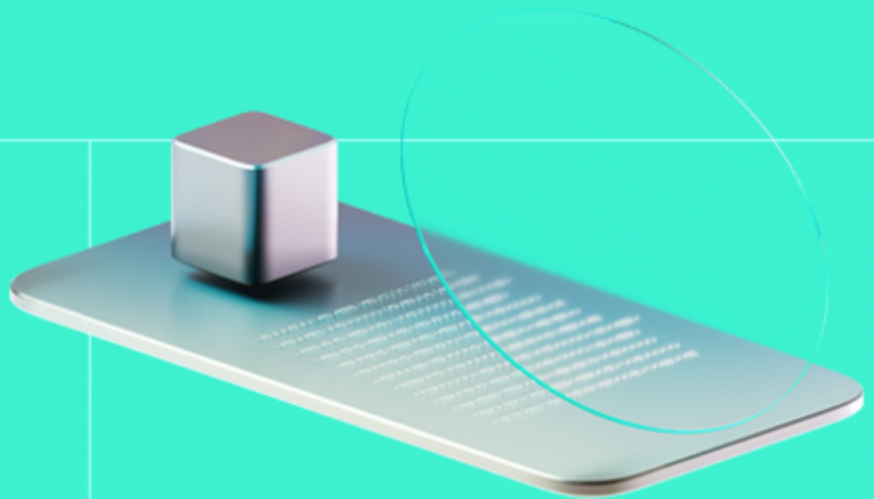




Smart Contract Code Review And Security Analysis Report

Customer: Astra Nova

Date: 06/11/2025



We express our gratitude to the Astra Nova team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

REVIVE Token is a **ERC20** token with custom burning functionality, transfer controls, and whitelisting for pre-TGE campaign.

Document

Name	Smart Contract Code Review and Security Analysis Report for Astra Nova
Audited By	Seher Saylik, Kornel Światłowski
Approved By	Ataberk Yavuzer
Website	https://astranova.world/
Changelog	24/10/2025 - Preliminary Report 06/11/2025 - Final Report
Platform	BSC
Language	Solidity
Tags	ERC20, Upgradable
Methodology	https://hacken.io/sc_methodology

Review Scope

Initial scope	https://bscscan.com/address/0xfb578047aba46ae0b20c0e074ac382e3817b7a9e#code
Repository	https://github.com/Astra-Nova/rvv-token-contracts-v1
Initial commit	373989d4ea6e07be8c6fdc2cfc05e2247793d19f
Remediation commit	5b5221f23c9407e5abbbdd75c271bc3e1d8e39886
Deployed contract	https://bscscan.com/address/0x03b6df017929f735eb3e8290ca282fa66e351226#code

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

6	5	0	1
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	0
Low	1

Vulnerability	Severity Status	
F-2025-13716 - setTransferAllowedTimestamp Logic Permanently Locks Timestamp Updates After 1 Day Past TGE	Low	Fixed
F-2025-13697 - Duplicate Access Control Implementations Increase Gas Costs	Info	Mitigated
F-2025-13700 - Unused onlyMultisig Modifier	Info	Fixed
F-2025-13702 - Mismatch Between NatSpec Description and Function Logic in burnFromBurnWallet	Info	Fixed
F-2025-13709 - Missing _disableInitializers() in Upgradable Contract Constructor	Info	Fixed
F-2025-13715 - Misleading NatSpec Comment on _transferAllowedTimestamp Parameter	Info	Fixed

Documentation quality

- Functional requirements are present, but only at a high-level.
 - Basic system description is provided.
 - Tokenomics is provided and roles are described in the documentation.
- Technical description is limited.
 - No run instructions.
 - No technical specification.
 - NatSpec is sufficient.

Code quality

- The development environment is configured.

Test coverage

Code coverage of the project is **94.12%** (branch coverage).

- Deployment and basic user interactions are covered with tests.
- Missing test run instructions.

Table of Contents

System Overview	6
Privileged Roles	6
Potential Risks	7
Findings	8
Vulnerability Details	8
Disclaimers	20
Appendix 1. Definitions	21
Severities	21
Potential Risks	21
Appendix 2. Scope	22
Appendix 3. Additional Valuables	23

System Overview

REVIVE Token is an upgradeable ERC20 token (symbol: RVV) designed to support a controlled pre-TGE (Token Generation Event) campaign. The contract mints a fixed supply of 10 billion tokens to a multisig treasury wallet upon initialization. The core functionality centers around time-gated transfer restrictions: before a configurable timestamp, only whitelisted addresses can transfer tokens, ensuring controlled distribution during the early campaign phase. After this timestamp expires, transfers become unrestricted for all holders.

Privileged roles

The **REVIVE Token** contract implements multiple privileged roles with distinct capabilities:

- **owner** - Inherited from **OwnableUpgradeable**, with administrative control over:
 - **Whitelist Management:** Add or remove addresses individually or in batches via `addToWhitelist()`, `removeFromWhitelist()`, and `batchSetWhitelisted()`
 - **Transfer Timing Control:** Modify the `transferAllowedTimestamp` through `setTransferAllowedTimestamp()`, with safeguards preventing extension beyond ETA (original timestamp + 1 day) after the initial period expires
- **treasuryWallet** - A designated multisig wallet (intended to be 4-of-6 Gnosis Safe):
 - **Token Custody:** Receives the entire initial supply of 10 billion tokens upon initialization
 - **Protected by onlyMultisig Modifier:** Though defined, this modifier is not currently utilized by any functions
- **burnWallet** - A dedicated address authorized for token destruction:
 - **Burn Authority:** Exclusively authorized to call `burnFromBurnWallet()` to burn tokens from its own balance
- **DEFAULT_ADMIN_ROLE** - Standard OpenZeppelin **AccessControlUpgradeable** role:
 - **Granted to Deployer:** Assigned to `msg.sender` during initialization
 - **Currently Unused:** No contract functions check for this role, suggesting it may be reserved for future upgrades

Potential Risks

- **Centralized Minting to a Single Address:** The project concentrates minting tokens in a single address, raising the risk of fund mismanagement or theft, especially if key storage security is compromised.
- **Flexibility and Risk in Contract Upgrades:** The project's contracts are upgradable, allowing the administrator to update the contract logic at any time. While this provides flexibility in addressing issues and evolving the project, it also introduces risks if upgrade processes are not properly managed or secured, potentially allowing for unauthorized changes that could compromise the project's integrity and security.
- **Absence of Upgrade Window Constraints:** The contract suite allows for immediate upgrades without a mandatory review or waiting period, increasing the risk of rapid deployment of malicious or flawed code, potentially compromising the system's integrity and user assets.

Findings

Vulnerability Details

F-2025-13716 - setTransferAllowedTimestamp Logic Permanently Locks Timestamp Updates After 1 Day Past TGE - Low

Description:

The function `setTransferAllowedTimestamp()` attempts to manage when transfers become globally allowed. However, its conditional logic introduces a hidden time lock that makes future timestamp updates impossible after a short window.

```
if (transferAllowedTimestamp > block.timestamp && ETA == 0) {
    transferAllowedTimestamp = newTimestamp;
} else {
    if (ETA == 0) {
        ETA = transferAllowedTimestamp + 1 days;
    }
    if (newTimestamp > ETA) revert InvalidTimestamp();
    transferAllowedTimestamp = newTimestamp;
}
```

When the function is called **after** the original TGE timestamp (i.e., `block.timestamp > transferAllowedTimestamp`), the first branch is skipped and `ETA` is set to `transferAllowedTimestamp + 1 days`.

This introduces an unintended restriction:

Any `newTimestamp` greater than `ETA` will revert.

Therefore, if more than one day has passed since the original TGE timestamp, **every possible future timestamp will exceed** `ETA`.

As a result, the function becomes permanently unusable for postponing the transfer date.

Assets:

- ReviveToken.sol
[<https://bscscan.com/address/0xfb578047aba46ae0b20c0e074ac382e3817b7a9e#code>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	4/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:	Simplify and correct the logic to prevent permanent lockout. Remove the <code>ETA</code> mechanism and enforce a clear forward-only rule.
Resolution:	The <code>setTransferAllowedTimestamp()</code> function is removed from the contract in the commit <code>def8766b</code> .

Evidences

PoC

Reproduce:

1. Initial deployment sets

`transferAllowedTimestamp = 1,000,000` and `ETA = 0`.

2. Time passes — now `block.timestamp = 1,100,000`
(which is more than one day after TGE).

3. The owner calls:

```
setTransferAllowedTimestamp(2,000,000);
```

4. Since `transferAllowedTimestamp > block.timestamp` is false,
the first `if` branch is skipped and the following code executes:

```
if (ETA == 0) {  
    ETA = transferAllowedTimestamp + 1 days;  
}
```

→ New `ETA = 1,000,000 + 86,400 = 1,086,400`.

5. The next line checks:

```
if (newTimestamp > ETA) revert InvalidTimestamp();
```

6. From this point forward, any future timestamp will still be greater than `ETA` (which is already smaller than `block.timestamp`), causing all future updates to revert and making the transfer date permanently unchangeable.

F-2025-13697 - Duplicate Access Control Implementations

Increase Gas Costs - Info

Description: The `ReviveToken` contract inherits from both the `OwnableUpgradeable` and `AccessControlUpgradeable` contracts provided by OpenZeppelin. Both of these modules are designed to manage access control within smart contracts, although they serve similar purposes through different mechanisms.

```
contract ReviveToken is Initializable, ERC20Upgradeable, OwnableUpgradeable,
    ReentrancyGuardUpgradeable, AccessControlUpgradeable {}
```

In the current implementation, access control logic based on the `OwnableUpgradeable` contract is actively used, as certain functions are protected by the `onlyOwner` modifier. However, the functionality from the `AccessControlUpgradeable` contract remains unused, aside from assigning the `DEFAULT_ADMIN_ROLE` in the `initialize()` function. No additional role-based logic or modifiers are applied in the remainder of the contract.

The presence of two distinct access control systems increases contract complexity, introduces redundant and unused logic, and results in unnecessary gas consumption during deployment. This design may also cause confusion regarding which access control system governs privileged actions and lead to inconsistent privilege management in future code extensions or upgrades.

Assets:

- `ReviveToken.sol`
[<https://bscscan.com/address/0xf578047aba46ae0b20c0e074ac382e3817b7a9e#code>]

Status: Mitigated

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

- Remediation:** It is recommended to remove the unused `AccessControlUpgradeable` inheritance and related role assignments if role-based logic is not required. Alternatively, the contract should consistently implement a single access control mechanism across all privileged operations to ensure clarity, maintainability, and efficient resource usage.
- Resolution:** The issue was mitigated in commit `66f8996`. Comments regarding the lack of usage of `AccessControlUpgradeable` were added to the code; however, the import remains to preserve the correct storage layout.

[F-2025-13700](#) - Unused onlyMultisig Modifier - Info

Description: The `ReviveToken` contract defines an `onlyMultisig()` modifier that is never utilized within the contract logic. The presence of unused code introduces unnecessary complexity and increases deployment gas costs. Additionally, redundant or unreferenced logic may lead to confusion regarding the contract's intended access control flow and create ambiguity for future maintenance or upgrades.

Assets:

- `ReviveToken.sol`
[<https://bscscan.com/address/0xfb578047aba46ae0b20c0e074ac382e3817b7a9e#code>]

Status: Fixed

Classification

Impact Rate: 1/5
Likelihood Rate: 5/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation: It is recommended to remove the unused `onlyMultisig()` modifier from the contract to reduce code complexity and deployment costs.

Resolution: The unused modifier is removed from the contract in the commit `2ccb0fe`.

[F-2025-13702](#) - Mismatch Between NatSpec Description and Function Logic in burnFromBurnWallet - Info

Description:

The `ReviveToken` contract defines the `burnFromBurnWallet()` function, which is responsible for burning a specified amount of tokens from the burn wallet. According to the function's NatSpec, it should only be callable by the entity assigned the `BURNER_ROLE`. However, the function does not enforce this restriction through role-based access control.

```
modifier onlyBurnWallet() {
    if (msg.sender != burnWallet) revert InvalidBurnerCaller();
    _;
}

/**
 * @notice Burns a specified amount of tokens from the burn wallet.
 * @dev Only callable by the designated burn wallet with the BURNER_ROLE.
 * @param _amount Total amount of tokens to burn.
 */
function burnFromBurnWallet(uint256 _amount) public onlyBurnWallet {
    if (_amount == 0) revert BurnAmountZero();
    if (balanceOf(burnWallet) < _amount) revert InsufficientBalance();
    _burn(burnWallet, _amount);
}
```

Instead, the function utilizes the `onlyBurnWallet` modifier, which validates that the `msg.sender` matches the address stored in the `burnWallet` variable. This approach diverges from the described access control mechanism in the NatSpec, resulting in a discrepancy between the documentation and the actual implementation.

Assets:

- `ReviveToken.sol`
[<https://bscscan.com/address/0xfb578047aba46ae0b20c0e074ac382e3817b7a9e#code>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: It is recommended to align the `burnFromBurnWallet()` function's NatSpec documentation with its actual implementation. Alternatively, the function can be modified to enforce the `BURNER_ROLE` access control to match the documented behavior. Consistency between the implemented logic and documented intent improves code clarity and maintainability.

Resolution: The NatSpec comment is corrected by the Astra Nova team in the commit `5488557` :

"@dev Only callable by the designated burn wallet address."

[F-2025-13709](#) - Missing `_disableInitializers()` in Upgradable Contract Constructor - Info

Description: OpenZeppelin's `Initializable` pattern supports secure upgradeable contract deployment by isolating initialization logic. For any contract meant to serve as a non-upgradable implementation, it is standard best practice to disable further initializations:

```
function _disableInitializers() internal virtual { ... }
```

This function prevents any subsequent call to `initialize` or similar initializer functions. The OpenZeppelin documentation explicitly recommends this safeguard for any implementation contracts that might be deployed behind a proxy.

Assets:

- `ReviveToken.sol`
[<https://bscscan.com/address/0xfb578047aba46ae0b20c0e074ac382e3817b7a9e#code>]

Status:

Fixed

Classification

Impact Rate: 1/5
Likelihood Rate: 5/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation: In each contract that inherits from `Initializable`, include the following in the constructor to prevent further initialization:

```
constructor() {  
    _disableInitializers();  
}
```

Resolution:

The initializer is disabled now in the commit [d39cdc0](#).

[F-2025-13715](#) - Misleading NatSpec Comment on `_transferAllowedTimestamp` Parameter - Info

Description:

The NatSpec comment for `_transferAllowedTimestamp` in the `initialize()` function states:

```
@param _transferAllowedTimestamp Timestamp after which transfers are allowed
for whitelisted users.
```

However, the implementation of `_update()` enforces the opposite condition:

```
if (block.timestamp < transferAllowedTimestamp && !whitelistedAddresses[from])
{
    revert AddressNotWhitelisted();
}
```

This means:

- Before `_transferAllowedTimestamp`: only whitelisted senders can transfer.
- After `_transferAllowedTimestamp`: everyone can transfer (no restriction).

So `_transferAllowedTimestamp` actually represents the time until which transfers are restricted, not “after which they are allowed only for whitelisted users.”

The existing comment suggests that whitelisted users become allowed only after that timestamp, which is the reverse of the real behavior.

Assets:

- `ReviveToken.sol`
[<https://bscscan.com/address/0xfb578047aba46ae0b20c0e074ac382e3817b7a9e#code>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Correct the NatSpec to accurately reflect runtime logic.

```
/**
 * @param _transferAllowedTimestamp The timestamp before which only whiteliste
 * d
 * addresses can transfer tokens. After this timestamp, transfers are
 * allowed for all users.
 */
```

Resolution: The NatSpec comment is fixed in the commit [04ae2ee](#) :
“@param *transferAllowedTimestamp* *Timestamp after which all transfers become unrestricted. Before this time, only whitelisted addresses can transfer.*”.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Initial scope	https://bscscan.com/address/0xfb578047aba46ae0b20c0e074ac382e3817b7a9e#code
Repository	https://github.com/Astra-Nova/rvv-token-contracts-v1
Initial commit	373989d4ea6e07be8c6fdc2cfc05e2247793d19f
Remediation commit	5b5221f23c9407e5abbbdd75c271bc3e1d8e39886
Whitepaper	N/A
Requirements	NatSpec
Technical Requirements	NatSpec
Deployed contract	https://bscscan.com/address/0x03b6df017929f735eb3e8290ca282fa66e351226#code

Asset	Type
ReviveToken.sol	Source
[https://bscscan.com/address/0xfb578047aba46ae0b20c0e074ac382e3817b7a9e#code] Code	

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 - Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.